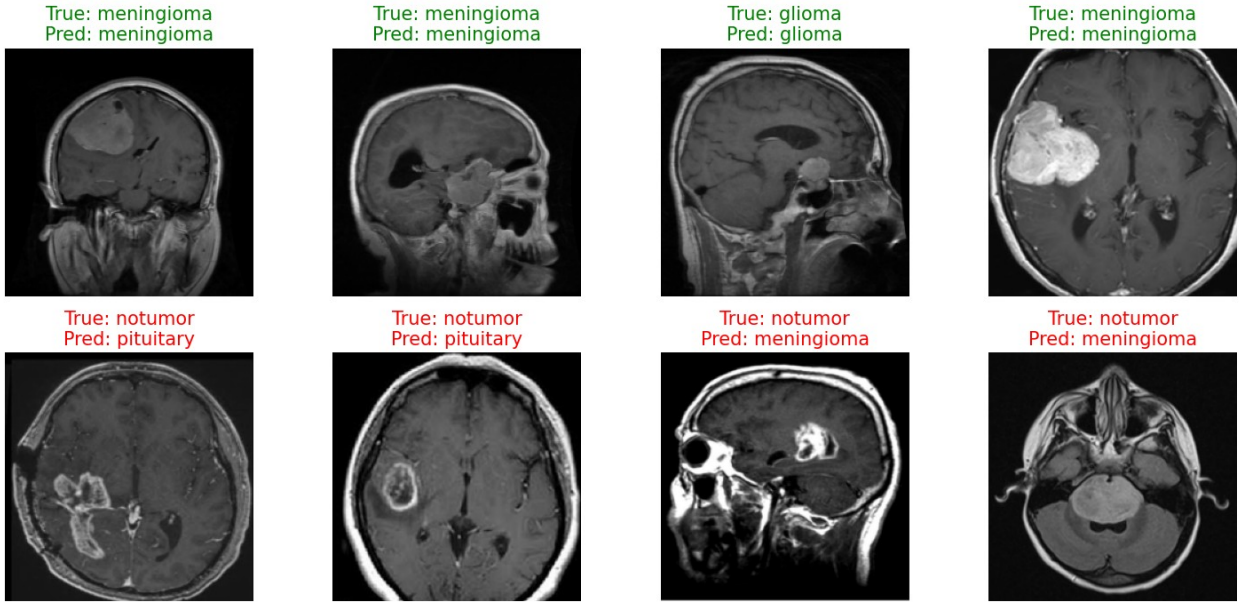
```
visualize_predictions(model2, test_loader, 8)
```



Model 3 : EfficientNet-V2-S

```
MEAN, STD = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]

effv2_train_transform = transforms.Compose([
    transforms.Resize((260, 260)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(p=0.2),
    transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.4, contrast=0.4,
saturation=0.2, hue=0.05),
    transforms.RandomAffine(degrees=10, translate=(0.1, 0.1),
shear=8),
    transforms.RandomGrayscale(p=0.1),
    transforms.RandAugment(num_ops=2, magnitude=9),   # extra
diversity
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
    transforms.RandomErasing(p=0.25, scale=(0.02, 0.15)),  # hides
random patch
])

effv2_train_dataset = ImageDataset(train_df,
transform=effv2_train_transform)
effv2_train_loader  = DataLoader(
    effv2_train_dataset,
    batch_size  = batch_size,
```

```
    shuffle     = True,
    num_workers = 2,
    pin_memory  = True
)

def mixup_data(images, labels, alpha=0.4):
    lam          = np.random.beta(alpha, alpha)
    random_index = torch.randperm(images.size(0)).to(images.device)
    mixed_images = lam * images + (1 - lam) * images[random_index]
    return mixed_images, labels, labels[random_index], lam

def mixup_loss(criterion, predictions, labels_a, labels_b, lam):
    return lam * criterion(predictions, labels_a) + \
           (1 - lam) * criterion(predictions, labels_b)

def build_efficientnet_v2s(num_classes=4):
    model = models.efficientnet_v2_s(weights='IMAGENET1K_V1')
    print("EfficientNet-V2-S loaded")

    # Freeze all layers first
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze last 3 blocks + classifier
    for name, param in model.named_parameters():
        if any(b in name for b in ['features.5', 'features.6',
'features.7', 'classifier']):
            param.requires_grad = True

    # Richer classifier head  (extra 512→256 block)
    in_features = model.classifier[1].in_features
    model.classifier = nn.Sequential(
        nn.Dropout(p=0.4),
        nn.Linear(in_features, 512),
        nn.SiLU(),
        nn.BatchNorm1d(512),
        nn.Dropout(p=0.3),
        nn.Linear(512, 256),
        nn.SiLU(),
        nn.BatchNorm1d(256),
        nn.Dropout(p=0.2),
        nn.Linear(256, num_classes),
    )
    return model.to(device)


model3 = build_efficientnet_v2s(num_classes=4)

trainable_params = sum(p.numel() for p in model3.parameters() if
p.requires_grad)
```

```
total_params     = sum(p.numel() for p in model3.parameters())
frozen_params    = total_params - trainable_params
print(f"\n Total parameters  : {total_params:,}")
print(f" Trainable (active): {trainable_params:,}")
print(f" Frozen (locked)   : {frozen_params:,}")
print(f" Training {trainable_params/total_params*100:.1f}% of the
model")
```

```
EfficientNet-V2-S loaded

 Total parameters  : 20,967,252
 Trainable (active): 19,145,676
 Frozen (locked)   : 1,821,576
 Training 91.3% of the model
```

```
BASE_LR = 2e-4
optimizer3 = optim.AdamW([
    {'params': [p for n, p in model3.named_parameters() if
'features.5' in n and p.requires_grad],
     'lr': BASE_LR * 0.05},   # very small — nearly frozen
    {'params': [p for n, p in model3.named_parameters() if
'features.6' in n and p.requires_grad],
     'lr': BASE_LR * 0.2},
    {'params': [p for n, p in model3.named_parameters() if
'features.7' in n and p.requires_grad],
     'lr': BASE_LR * 0.5},
    {'params': model3.classifier.parameters(),
     'lr': BASE_LR},           # highest LR for the new head
], weight_decay=2e-4)

criterion3 = nn.CrossEntropyLoss(label_smoothing=0.1)

scheduler3 = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
    optimizer3, T_0=10, T_mult=2, eta_min=1e-7
)

BEST_MODEL_PATH = 'best_brain_tumor_effv2s.pth'

def train_effv2s(model, train_loader, val_loader,
                 criterion, optimizer, scheduler,
                 num_epochs=60, patience=20):

    best_val_acc = 0.0
    no_improve   = 0
    history      = {'train_loss': [], 'train_acc': [],
                    'val_loss':   [], 'val_acc':   []}

    print(f"\n{'='*70}")
    print(f" Training: EfficientNet-V2-S | Device: {device}")
    print(f" Epochs: {num_epochs} | Patience: {patience}")
```

```
    print(f"{'='*70}\n")

    for epoch in range(num_epochs):

        # ── Train ──────────────────────────────────────────────
        model.train()
        running_loss, correct_train, total_train = 0.0, 0, 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            if np.random.random() > 0.5:   # Mixup 50 % of time
                images, labels_a, labels_b, lam = mixup_data(images,
labels, alpha=0.4)
                optimizer.zero_grad()
                outputs = model(images)
                loss    = mixup_loss(criterion, outputs, labels_a,
labels_b, lam)
            else:
                optimizer.zero_grad()
                outputs = model(images)
                loss    = criterion(outputs, labels)

            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
            optimizer.step()

            running_loss  += loss.item()
            _, predicted   = torch.max(outputs, 1)
            total_train   += labels.size(0)
            correct_train += (predicted == labels).sum().item()

        train_loss = running_loss / len(train_loader)
        train_acc  = 100.0 * correct_train / total_train

        # ── Validate ───────────────────────────────────────────
        model.eval()
        val_loss, correct_val, total_val = 0.0, 0, 0

        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                val_loss   += criterion(outputs, labels).item()
                _, predicted = torch.max(outputs, 1)
                total_val   += labels.size(0)
                correct_val += (predicted == labels).sum().item()
```

```
        val_loss = val_loss / len(val_loader)
        val_acc  = 100.0 * correct_val / total_val

        scheduler.step(epoch + val_acc / 100)

        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)

        current_lr = optimizer.param_groups[-1]['lr']
        print(f"Epoch [{epoch+1}/{num_epochs}] | LR:
{current_lr:.6f}")
        print(f"Train Loss: {train_loss:.4f}, Train Accuracy:
{train_acc:.2f}%")
        print(f"Val   Loss: {val_loss:.4f}, Val   Accuracy:
{val_acc:.2f}%")
        print("#" * 80)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            no_improve   = 0
            torch.save(model.state_dict(), BEST_MODEL_PATH)
            print(f" Best model saved (val_acc: {val_acc:.2f}%)")
        else:
            no_improve += 1
            print(f" No improvement. Patience:
{no_improve}/{patience}")

        if train_acc >= 99.5 and val_acc >= 99.5:
            print(f"\n 99.5% target reached — stopping.")
            break

        if no_improve >= patience:
            print(f"\n Early stopping after {epoch+1} epochs.")
            break

    print(f"\n Best Validation Accuracy: {best_val_acc:.2f}%")
    return history

print("\n" + "-"*70)
print(" MODEL 3 : EfficientNet-V2-S")
print("-"*70)
print("Starting training...")

history3 = train_effv2s(
    model        = model3,
    train_loader = effv2_train_loader,
    val_loader   = val_loader,
    criterion    = criterion3,
```

```
    optimizer    = optimizer3,
    scheduler    = scheduler3,
    num_epochs   = 60,
    patience     = 20,
)


----------------------------------------------------------------------
 MODEL 3 : EfficientNet-V2-S
----------------------------------------------------------------------
Starting training...

======================================================================
 Training: EfficientNet-V2-S | Device: cuda
 Epochs: 60 | Patience: 20
======================================================================

Epoch [1/60] | LR: 0.000197
Train Loss: 0.9547, Train Accuracy: 61.34%
Val   Loss: 0.7202, Val   Accuracy: 81.88%
######################################################################
##########
 Best model saved (val_acc: 81.88%)
Epoch [2/60] | LR: 0.000184
Train Loss: 0.8007, Train Accuracy: 67.27%
Val   Loss: 0.6820, Val   Accuracy: 83.88%
######################################################################
##########
 Best model saved (val_acc: 83.88%)
Epoch [3/60] | LR: 0.000162
Train Loss: 0.7469, Train Accuracy: 71.09%
Val   Loss: 0.6487, Val   Accuracy: 86.12%
######################################################################
##########
 Best model saved (val_acc: 86.12%)
Epoch [4/60] | LR: 0.000134
Train Loss: 0.6846, Train Accuracy: 75.12%
Val   Loss: 0.5841, Val   Accuracy: 90.38%
######################################################################
##########
 Best model saved (val_acc: 90.38%)
Epoch [5/60] | LR: 0.000102
Train Loss: 0.6729, Train Accuracy: 75.79%
Val   Loss: 0.5358, Val   Accuracy: 92.75%
######################################################################
##########
 Best model saved (val_acc: 92.75%)
Epoch [6/60] | LR: 0.000071
Train Loss: 0.6690, Train Accuracy: 76.41%
Val   Loss: 0.5305, Val   Accuracy: 92.50%
```

```
#################################################################
##########
 No improvement. Patience: 1/20
Epoch [7/60] | LR: 0.000043
Train Loss: 0.6191, Train Accuracy: 76.45%
Val   Loss: 0.5383, Val   Accuracy: 92.00%
#################################################################
##########
 No improvement. Patience: 2/20
Epoch [8/60] | LR: 0.000021
Train Loss: 0.6480, Train Accuracy: 77.29%
Val   Loss: 0.5418, Val   Accuracy: 91.38%
#################################################################
##########
 No improvement. Patience: 3/20
Epoch [9/60] | LR: 0.000006
Train Loss: 0.6206, Train Accuracy: 79.00%
Val   Loss: 0.5376, Val   Accuracy: 92.12%
#################################################################
##########
 No improvement. Patience: 4/20
Epoch [10/60] | LR: 0.000000
Train Loss: 0.6404, Train Accuracy: 76.95%
Val   Loss: 0.5195, Val   Accuracy: 92.12%
#################################################################
##########
 No improvement. Patience: 5/20
Epoch [11/60] | LR: 0.000199
Train Loss: 0.6579, Train Accuracy: 76.32%
Val   Loss: 0.5375, Val   Accuracy: 92.12%
#################################################################
##########
 No improvement. Patience: 6/20
Epoch [12/60] | LR: 0.000195
Train Loss: 0.6467, Train Accuracy: 77.18%
Val   Loss: 0.5289, Val   Accuracy: 92.75%
#################################################################
##########
 No improvement. Patience: 7/20
Epoch [13/60] | LR: 0.000190
Train Loss: 0.6543, Train Accuracy: 79.55%
Val   Loss: 0.4956, Val   Accuracy: 93.75%
#################################################################
##########
 Best model saved (val_acc: 93.75%)
Epoch [14/60] | LR: 0.000182
Train Loss: 0.5971, Train Accuracy: 78.61%
Val   Loss: 0.5040, Val   Accuracy: 93.38%
#################################################################
```

```
##########
 No improvement. Patience: 1/20
Epoch [15/60] | LR: 0.000172
Train Loss: 0.6129, Train Accuracy: 78.25%
Val   Loss: 0.5131, Val   Accuracy: 92.50%
##################################################################
##########
 No improvement. Patience: 2/20
Epoch [16/60] | LR: 0.000160
Train Loss: 0.6016, Train Accuracy: 78.43%
Val   Loss: 0.5166, Val   Accuracy: 93.12%
##################################################################
##########
 No improvement. Patience: 3/20
Epoch [17/60] | LR: 0.000146
Train Loss: 0.5761, Train Accuracy: 79.14%
Val   Loss: 0.4608, Val   Accuracy: 95.38%
##################################################################
##########
 Best model saved (val_acc: 95.38%)
Epoch [18/60] | LR: 0.000132
Train Loss: 0.5916, Train Accuracy: 80.41%
Val   Loss: 0.4771, Val   Accuracy: 94.88%
##################################################################
##########
 No improvement. Patience: 1/20
Epoch [19/60] | LR: 0.000117
Train Loss: 0.5773, Train Accuracy: 80.02%
Val   Loss: 0.4799, Val   Accuracy: 94.38%
##################################################################
##########
 No improvement. Patience: 2/20
Epoch [20/60] | LR: 0.000101
Train Loss: 0.5762, Train Accuracy: 79.95%
Val   Loss: 0.4695, Val   Accuracy: 95.25%
##################################################################
##########
 No improvement. Patience: 3/20
Epoch [21/60] | LR: 0.000085
Train Loss: 0.5428, Train Accuracy: 81.07%
Val   Loss: 0.4514, Val   Accuracy: 95.62%
##################################################################
##########
 Best model saved (val_acc: 95.62%)
Epoch [22/60] | LR: 0.000070
Train Loss: 0.5718, Train Accuracy: 77.93%
Val   Loss: 0.4687, Val   Accuracy: 95.00%
##################################################################
##########
```

```
 No improvement. Patience: 1/20
Epoch [23/60] | LR: 0.000055
Train Loss: 0.5491, Train Accuracy: 80.11%
Val   Loss: 0.4793, Val   Accuracy: 95.38%
#################################################################
##########
 No improvement. Patience: 2/20
Epoch [24/60] | LR: 0.000042
Train Loss: 0.5499, Train Accuracy: 81.82%
Val   Loss: 0.4488, Val   Accuracy: 96.00%
#################################################################
##########
 Best model saved (val_acc: 96.00%)
Epoch [25/60] | LR: 0.000030
Train Loss: 0.5446, Train Accuracy: 82.11%
Val   Loss: 0.4525, Val   Accuracy: 96.00%
#################################################################
##########
 No improvement. Patience: 1/20
Epoch [26/60] | LR: 0.000020
Train Loss: 0.5698, Train Accuracy: 80.61%
Val   Loss: 0.4552, Val   Accuracy: 95.62%
#################################################################
##########
 No improvement. Patience: 2/20
Epoch [27/60] | LR: 0.000011
Train Loss: 0.5291, Train Accuracy: 80.62%
Val   Loss: 0.4531, Val   Accuracy: 95.62%
#################################################################
##########
 No improvement. Patience: 3/20
Epoch [28/60] | LR: 0.000005
Train Loss: 0.5247, Train Accuracy: 81.48%
Val   Loss: 0.4552, Val   Accuracy: 95.25%
#################################################################
##########
 No improvement. Patience: 4/20
Epoch [29/60] | LR: 0.000001
Train Loss: 0.5760, Train Accuracy: 81.00%
Val   Loss: 0.4690, Val   Accuracy: 95.50%
#################################################################
##########
 No improvement. Patience: 5/20
Epoch [30/60] | LR: 0.000000
Train Loss: 0.5489, Train Accuracy: 80.05%
Val   Loss: 0.4642, Val   Accuracy: 95.50%
#################################################################
##########
 No improvement. Patience: 6/20
```

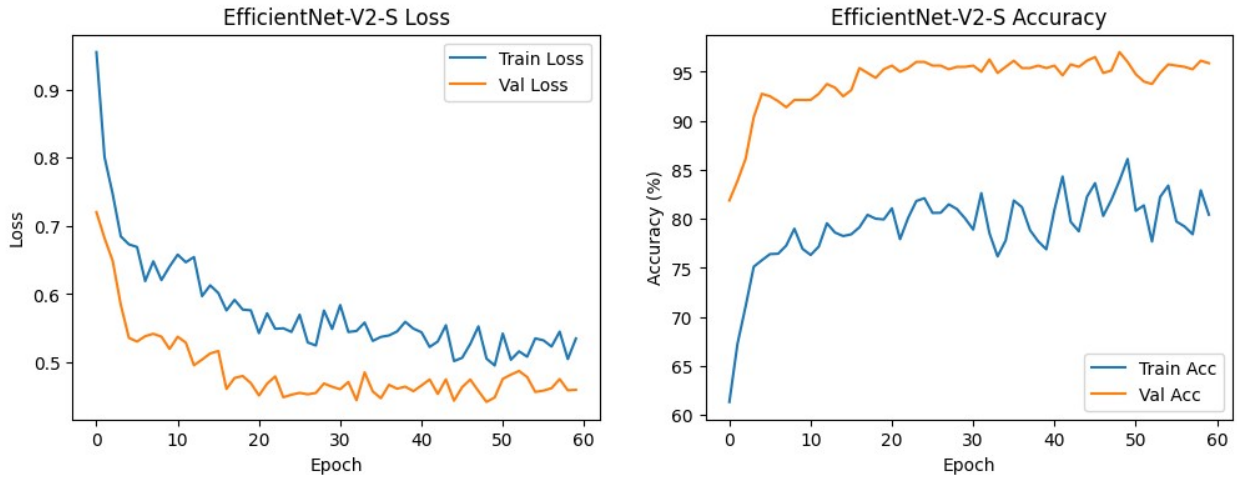
```
Epoch [31/60] | LR: 0.000200
Train Loss: 0.5839, Train Accuracy: 78.91%
Val   Loss: 0.4606, Val   Accuracy: 95.62%
#####################################################################
##########
 No improvement. Patience: 7/20
Epoch [32/60] | LR: 0.000199
Train Loss: 0.5445, Train Accuracy: 82.62%
Val   Loss: 0.4711, Val   Accuracy: 95.00%
#####################################################################
##########
 No improvement. Patience: 8/20
Epoch [33/60] | LR: 0.000197
Train Loss: 0.5459, Train Accuracy: 78.54%
Val   Loss: 0.4444, Val   Accuracy: 96.25%
#####################################################################
##########
 Best model saved (val_acc: 96.25%)
Epoch [34/60] | LR: 0.000195
Train Loss: 0.5583, Train Accuracy: 76.18%
Val   Loss: 0.4853, Val   Accuracy: 94.88%
#####################################################################
##########
 No improvement. Patience: 1/20
Epoch [35/60] | LR: 0.000193
Train Loss: 0.5314, Train Accuracy: 77.80%
Val   Loss: 0.4573, Val   Accuracy: 95.50%
#####################################################################
##########
 No improvement. Patience: 2/20
Epoch [36/60] | LR: 0.000189
Train Loss: 0.5373, Train Accuracy: 81.88%
Val   Loss: 0.4473, Val   Accuracy: 96.12%
#####################################################################
##########
 No improvement. Patience: 3/20
Epoch [37/60] | LR: 0.000185
Train Loss: 0.5394, Train Accuracy: 81.20%
Val   Loss: 0.4670, Val   Accuracy: 95.38%
#####################################################################
##########
 No improvement. Patience: 4/20
Epoch [38/60] | LR: 0.000181
Train Loss: 0.5453, Train Accuracy: 78.84%
Val   Loss: 0.4613, Val   Accuracy: 95.38%
#####################################################################
##########
 No improvement. Patience: 5/20
Epoch [39/60] | LR: 0.000176
```

```
Train Loss: 0.5594, Train Accuracy: 77.71%
Val   Loss: 0.4643, Val   Accuracy: 95.62%
#################################################################
##########
 No improvement. Patience: 6/20
Epoch [40/60] | LR: 0.000171
Train Loss: 0.5494, Train Accuracy: 76.89%
Val   Loss: 0.4577, Val   Accuracy: 95.38%
#################################################################
##########
 No improvement. Patience: 7/20
Epoch [41/60] | LR: 0.000165
Train Loss: 0.5440, Train Accuracy: 80.95%
Val   Loss: 0.4661, Val   Accuracy: 95.62%
#################################################################
##########
 No improvement. Patience: 8/20
Epoch [42/60] | LR: 0.000159
Train Loss: 0.5225, Train Accuracy: 84.32%
Val   Loss: 0.4748, Val   Accuracy: 94.62%
#################################################################
##########
 No improvement. Patience: 9/20
Epoch [43/60] | LR: 0.000153
Train Loss: 0.5306, Train Accuracy: 79.70%
Val   Loss: 0.4536, Val   Accuracy: 95.75%
#################################################################
##########
 No improvement. Patience: 10/20
Epoch [44/60] | LR: 0.000146
Train Loss: 0.5544, Train Accuracy: 78.73%
Val   Loss: 0.4750, Val   Accuracy: 95.50%
#################################################################
##########
 No improvement. Patience: 11/20
Epoch [45/60] | LR: 0.000139
Train Loss: 0.5018, Train Accuracy: 82.25%
Val   Loss: 0.4438, Val   Accuracy: 96.12%
#################################################################
##########
 No improvement. Patience: 12/20
Epoch [46/60] | LR: 0.000131
Train Loss: 0.5066, Train Accuracy: 83.64%
Val   Loss: 0.4636, Val   Accuracy: 96.50%
#################################################################
##########
 Best model saved (val_acc: 96.50%)
Epoch [47/60] | LR: 0.000124
Train Loss: 0.5269, Train Accuracy: 80.30%
```

```
Val   Loss: 0.4748, Val   Accuracy: 94.88%
##################################################################
##########
 No improvement. Patience: 1/20
Epoch [48/60] | LR: 0.000116
Train Loss: 0.5527, Train Accuracy: 81.93%
Val   Loss: 0.4580, Val   Accuracy: 95.12%
##################################################################
##########
 No improvement. Patience: 2/20
Epoch [49/60] | LR: 0.000108
Train Loss: 0.5054, Train Accuracy: 83.89%
Val   Loss: 0.4419, Val   Accuracy: 97.00%
##################################################################
##########
 Best model saved (val_acc: 97.00%)
Epoch [50/60] | LR: 0.000100
Train Loss: 0.4954, Train Accuracy: 86.11%
Val   Loss: 0.4485, Val   Accuracy: 96.00%
##################################################################
##########
 No improvement. Patience: 1/20
Epoch [51/60] | LR: 0.000093
Train Loss: 0.5422, Train Accuracy: 80.82%
Val   Loss: 0.4758, Val   Accuracy: 94.75%
##################################################################
##########
 No improvement. Patience: 2/20
Epoch [52/60] | LR: 0.000085
Train Loss: 0.5038, Train Accuracy: 81.38%
Val   Loss: 0.4819, Val   Accuracy: 94.00%
##################################################################
##########
 No improvement. Patience: 3/20
Epoch [53/60] | LR: 0.000077
Train Loss: 0.5161, Train Accuracy: 77.70%
Val   Loss: 0.4873, Val   Accuracy: 93.75%
##################################################################
##########
 No improvement. Patience: 4/20
Epoch [54/60] | LR: 0.000070
Train Loss: 0.5083, Train Accuracy: 82.23%
Val   Loss: 0.4784, Val   Accuracy: 94.88%
##################################################################
##########
 No improvement. Patience: 5/20
Epoch [55/60] | LR: 0.000062
Train Loss: 0.5351, Train Accuracy: 83.39%
Val   Loss: 0.4566, Val   Accuracy: 95.75%
```

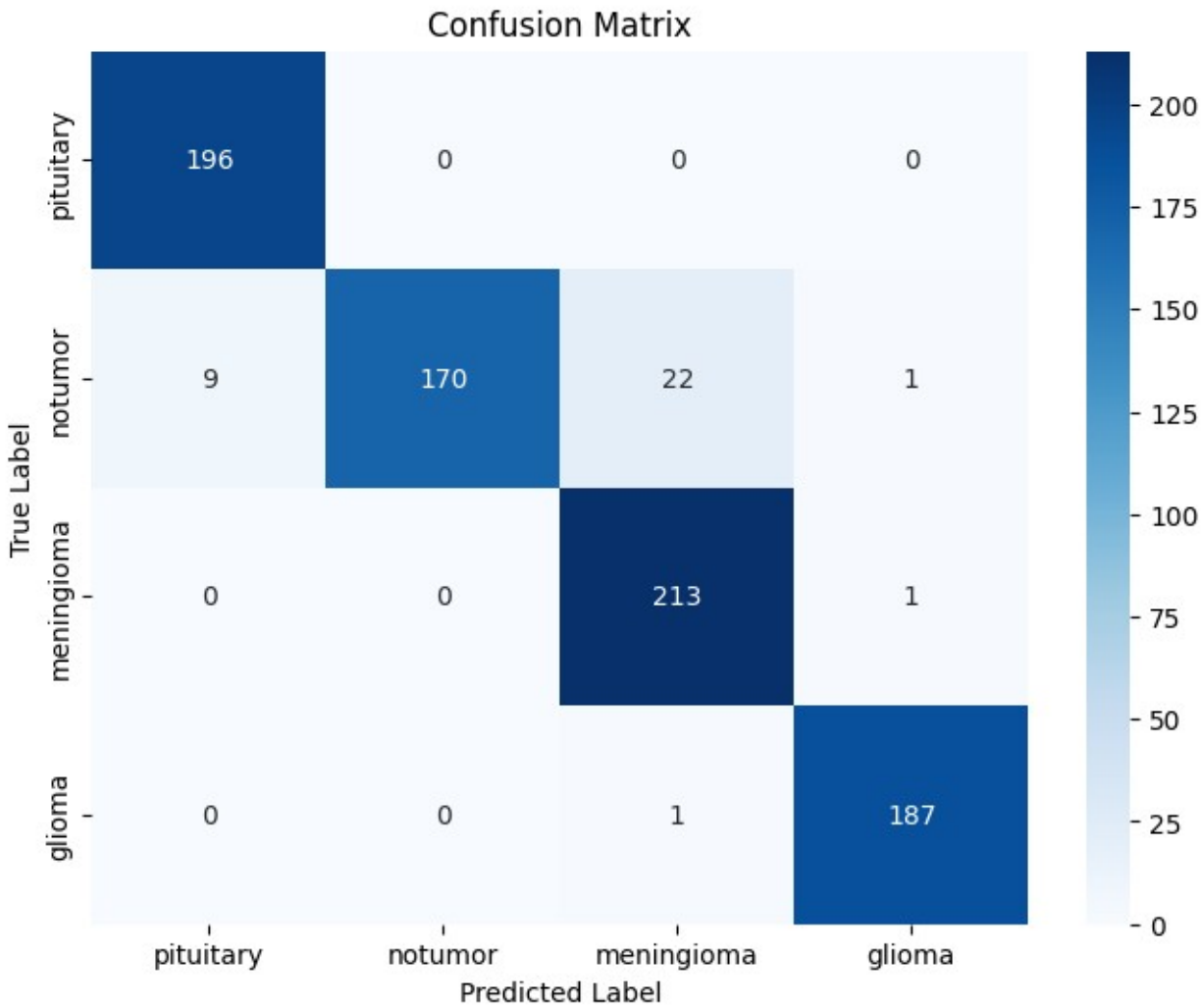
```
##################################################################
##########
 No improvement. Patience: 6/20
Epoch [56/60] | LR: 0.000055
Train Loss: 0.5323, Train Accuracy: 79.73%
Val   Loss: 0.4585, Val   Accuracy: 95.62%
##################################################################
##########
 No improvement. Patience: 7/20
Epoch [57/60] | LR: 0.000048
Train Loss: 0.5231, Train Accuracy: 79.23%
Val   Loss: 0.4621, Val   Accuracy: 95.50%
##################################################################
##########
 No improvement. Patience: 8/20
Epoch [58/60] | LR: 0.000042
Train Loss: 0.5450, Train Accuracy: 78.43%
Val   Loss: 0.4754, Val   Accuracy: 95.25%
##################################################################
##########
 No improvement. Patience: 9/20
Epoch [59/60] | LR: 0.000035
Train Loss: 0.5047, Train Accuracy: 82.91%
Val   Loss: 0.4590, Val   Accuracy: 96.12%
##################################################################
##########
 No improvement. Patience: 10/20
Epoch [60/60] | LR: 0.000030
Train Loss: 0.5349, Train Accuracy: 80.43%
Val   Loss: 0.4597, Val   Accuracy: 95.88%
##################################################################
##########
 No improvement. Patience: 11/20

 Best Validation Accuracy: 97.00%
```

```
plot_training_history(history3, model_name="EfficientNet-V2-S")
```

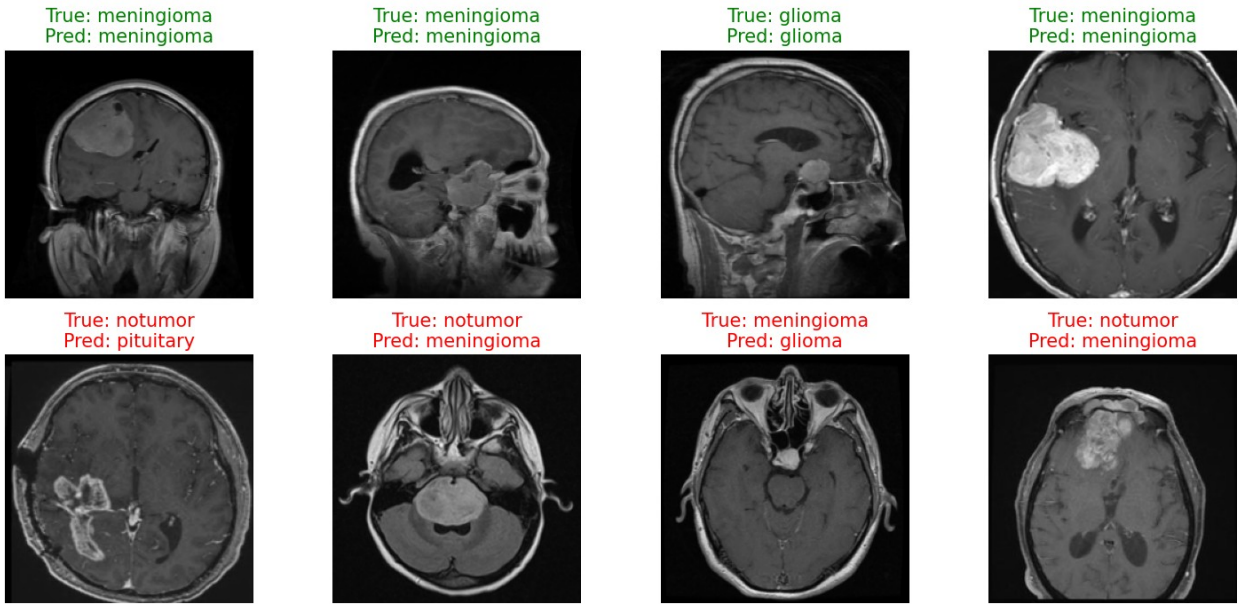



```python
print("\nLoading best model for testing...")
model3.load_state_dict(torch.load(BEST_MODEL_PATH))
test_model(model3, test_loader)
```

```
Loading best model for testing...
Test Accuracy: 95.75%

Classification Report:

              precision    recall  f1-score   support

   pituitary       0.96      1.00      0.98       196
     notumor       1.00      0.84      0.91       202
  meningioma       0.90      1.00      0.95       214
      glioma       0.99      0.99      0.99       188

    accuracy                           0.96       800
   macro avg       0.96      0.96      0.96       800
weighted avg       0.96      0.96      0.96       800
```




```
visualize_predictions(model3, test_loader, 8)
```




```
DEPLOY_PATH = 'brain_tumor_effv2s_final.pth'
torch.save({
    'model_name' : 'efficientnet_v2_s',
    'num_classes': num_classes,
    'categories' : categories,
    'label_map'  : label_map,
    'state_dict' : model3.state_dict(),
}, DEPLOY_PATH)
print(f"\nFinal deployment model saved → {DEPLOY_PATH}")
```

```
Final deployment model saved → brain_tumor_effv2s_final.pth
```

```
def predict_single_image(image_path):
    checkpoint   = torch.load(DEPLOY_PATH, map_location=device)
    loaded_model =
build_efficientnet_v2s(num_classes=checkpoint['num_classes'])
    loaded_model.load_state_dict(checkpoint['state_dict'])
    loaded_model.eval()
    class_names = checkpoint['categories']

    preprocess = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=MEAN, std=STD),
    ])

    image      = Image.open(image_path).convert('RGB')
    tensor     = preprocess(image).unsqueeze(0).to(device)

    with torch.no_grad():
```

```
        probs = torch.softmax(loaded_model(tensor),
dim=1).squeeze().cpu().numpy()

    predicted_idx = np.argmax(probs)

    print(f"\n{'='*45}")
    print(f" Predicted Class : {class_names[predicted_idx]}")
    print(f" Confidence      : {probs[predicted_idx]*100:.1f}%")
    print(f"{'='*45}")
    print("\n All class probabilities:")
    for name, prob in zip(class_names, probs):
        bar = '█' * int(prob * 40)
        print(f"  {name:<12} {bar}  {prob*100:.1f}%")

    plt.figure(figsize=(4, 4))
    plt.imshow(image, cmap='gray')
    plt.title(f"Predicted: {class_names[predicted_idx]}\n"
              f"Confidence: {probs[predicted_idx]*100:.1f}%",
fontsize=12)
    plt.axis('off')
    plt.tight_layout()
    plt.show()

    return class_names[predicted_idx], probs

predict_single_image()

predict_single_image()
```